

כמה ילדים ה-API של המורה מחזיק — לפני ואחרי השינויים

סיכום בדיקות העומס של backend-ai-tutor-he, 25.09.2026. נמדד על השרת הנוכחי (2 ליבות, 2GB, ניו יורק) עם עותק נפרד של השרת, בסיס נתונים מדומה וגם בסיס הנתונים האמיתי (קריאה בלבד, עם הגבלה). המטרה שהוגדרה: יותר מ-100,000 ילדים בו-זמנית.



השורה התחתונה

- לפני: תהליך אחד של השרת החזיק 155 ילדים בשיעור. פתיחת שיעור ע"י 8 ילדים יחד הקפיאה את כל השרת לחצי שנייה, והצ'אט הראשי הקפיא את כל שאר הילדים עד 14 שניות תחת עומס.
- אחרי: כ-177 ילדים לליבה, 54% פחות קריאות לבסיס הנתונים לכל ילד, והנתיבים שמסך השיעור שואל שוב ושוב עונים פי 2.5 עד 6 יותר בקשות בשנייה, בשליש מהזמן.
- השרת גדל עם הליבות: מעבר מליבה אחת לשתיים נתן +58% באותם נתיבים. כל תהליך נוסף עולה כ-170MB זיכרון.
- 100,000 ילדים בו-זמנית זה כ-560 ליבות עסוקות, כ-21,000 קריאות לבסיס הנתונים בשנייה ואלפי קריאות למודלים בשנייה. זה לא שרת אחד גדול יותר — זו ארכיטקטורה אחרת (עמוד 4). בגודל כזה החשבון על המודלים, לא השרתים, הוא התקרה.
- העבודה הכבדה צריכה לצאת מהמכונה שהילדים מדברים איתה: קודם ה-worker של המדיה (בלי שינוי קוד), אחר כך נתיבי ה-AI לקבוצת שרתים משלהם.

מה שונה בקוד (עדיין לא הועלה — מחכה לאישור)

שינוי	איפה	מה זה נתן
בדיקת הכניסה (token) מקומית מול המפתח הציבורי של הפרויקט במקום פנייה ל-Supabase	request_cache.py, authenticate_user	קריאה אחת פחות לבסיס הנתונים בכל בקשה
שורת הילד נשמרת 60 שני' (נמחקת בעדכון פרטי ילד)	get_child_by_id	עוד קריאה אחת פחות כמעט בכל בקשה
מנוי 60 שני', שורת תוכנית לימודים 5 דק'	is_paid_active_subscription, get_learning_lesson	פתיחת שיעור: 10 ← 3 קריאות
אותה משימת מדיה לא נשלחת שוב תוך 60 שני'	enqueue_media_job	פתיחת שיעור לא שולחת 3 משימות בכל בדיקה
3 נתיבים כבר לא מחכים לרשת בתוך לולאת השרת	openai_clean_chat, homework_coach_v2, unit-lesson	פתיחת שיעור 0.7 ← 25.7 בקשות/שני'; הצ'אט לא מקפיא

המחיר היחיד: אחרי התנתקות, כניסה נשארת בתוקף עד שה-token פג (עד שעה) — כמו בכל לקוח Supabase שבודק מקומית. מבטל את כל המטמונים REQUEST_CACHE=0, מבטל את זה AUTH_LOCAL_JWT=0

לפני ← אחרי, במספרים

זמן טיפול (latency) ותפוקה לכל נתיב. p50 = הזמן שחצי מהבקשות עומדות בו, 95% = p95 מהבקשות.

בסיס הנתונים האמיתי (אותם 6 נתיבי קריאה, אותו שרת, UTC 06:45-06:37)

בכל תא מספרי: לפני → אחרי.

נתיב	משתמש אחד, מ"ש	בקשות/שנ' ב-8 משתמשים	ב-8 משתמשים p95 מ"ש	לבקשה CPU, מ"ש	קריאות DB לבקשה
units	352 → 122	19.2 → 52.9	798 → 257	17.1 → 11.0	3 → 1
active-lesson-state	363 → 120	18.0 → 53.3	1,463 → 418	23.3 → 14.4	3 → 1
audio	487 → 132	15.1 → 38.0	722 → 309	28.6 → 19.9	4 → 1
check-set	234 → 5	30.6 → 142.2	363 → 230	13.6 → 6.3	2 → 0
kid-lessons	347 → 118	21.1 → 53.0	514 → 274	15.5 → 10.4	3 → 1
kid-get	222 → 5	26.8 → 170.0	529 → 98	14.4 → 5.3	2 → 0

בקוד הישן התהליך עמד על 30%-39% CPU בזמן שהתפוקה נשארה שטוחה: הוא חיכה לנסיעות הלוך-חזור ל-Supabase. בקוד החדש אותם נתיבים מביאים את המעבד ל-50%-95 — המגבלה היא עכשיו המכונה, ואת זה אפשר לקנות. השירות החי נשאר בחציון 11-12 מ"ש בשתי הבדיקות; משתמשי הבדיקה נמחקו.

כל סוגי הנתיבים (בסיס נתונים מדומה, 100 מ"ש לקריאה, זמני מודלים כמו בפרודקשן)

בכל תא מספרי: לפני → אחרי.

נתיב	סוג	בקשות/שנ' בשיא תקין	לבקשה, מ"ש CPU	קריאות DB	השהיית כל השרת (מקס'), מ"ש
unit-lesson	media	0.7 → 25.7	72 → 42	10 → 3	537 → 457
visuals	read	15.7 → 37.5	38 → 20	4 → 1	258 → 441
audio	media	14.4 → 39.3	34 → 20	4 → 1	179 → 516
hero-image	media	16.1 → 40.8	36 → 22	4 → 1	312 → 631
tts	model	42.3 → 115.8	22 → 9	2 → 0	1,168 → 410
kid-get	read	34.7 → 181.3	14 → 5	2 → 0	198 → 576
check-set	read	35.1 → 160.2	14 → 6	2 → 0	197 → 487
structured-lesson	model	1.1 → 4.2	109 → 126	13 → 10	200 → 731
tutor-chat	model	4.3 → 4.5	102 → 93	8 → 6	518 → 571
openai-clean-chat	model	-	60 → 80	2 → 0	14,104 → 25

"השהיית כל השרת" = כמה זמן בקשה פשוטה של ילד אחר חיכתה בזמן העומס. בצ'אט הראשי זה ירד מ-14 שניות ל-25 מ"ש ב-256 משתמשים בו-זמנית (התפוקה שלו לא נמדדה: המודל המדומה עונה אחרי 20 שנ', יותר משלב הבדיקה).

זמני טיפול בפרודקשן (מידות קודמות, 14-16.09, מתוך ai_calls ויומני השרת)

פעולה	זמן	הערה
תשובת צ'אט של המורה	4.5-6 שניות (חציון)	רובו המודל
יצירת שיעור חדש (טקסט)	70-92 שניות	לבד 14-46 שנ' Visual Director
קול ראשון בשיעור חדש	כ-107 שניות	הקול מוגש רק כשכל החלקים מוכנים
משימת מדיה (תמונות + קול)	68-339 שניות	ב-worker, לא בבקשה של הילד
קריאה פשוטה ל-DB (אחרי השינוי)	120-130 מ"ש	רובו נסיעה הלוך-חזור לאזור של Supabase
נתיב שכבר במטמון (אחרי השינוי)	5 מ"ש	check-set, kid-get

כמה משתמשים, על איזו מכונה, ובכמה

ילד אחד בשיעור בדקה (לפי תמהיל הבקשות של מסך השיעור): 4 בדיקות סטטוס, תשובה אחת, חצי הודעת צ'אט, קריאת טקסט בקול ועוד.

לפני	אחרי	
270 מ"ש	238 מ"ש	זמן מעבד של השרת לילד בדקה
27.7	12.8 (-54%)	קריאות לבסיס הנתונים לילד בדקה
155	177	ילדים לכל ליבה (70% ניצול)

זמן המעבד לילד ירד פחות מהנתיבים עצמם, כי דקה של ילד נשלטת ע"י נתיבי ה-AI (תשובות בשיעור, צ'אט) — 80-126 מ"ש של המעבד שלנו לכל קריאה, בערך 3-4 מ"ש לכל קריאת רשת יוצאת כפול כ-10 קריאות DB וקריאת מודל. זה המנוף הבא.

ליבות וזיכרון — נמדד

ליבה אחת, תהליך אחד	2 ליבות, 2 תהליכים	
222 בקשות/שני'	349 בקשות/שני' (+58%)	בשיא kid-get
194 בקשות/שני'	277 בקשות/שני' (+43%)	בשיא check-set
178MB	348MB (170-כ-MB לתהליך)	זיכרון (RSS)

במידה עם 2 ליבות, מחולל העומס, השרתים המדומים והפרודקשן חלקו את אותן ליבות — במכונה עם ליבות פנויות הרווח לליבה גבוה יותר.

מה כל מכונה תחזיק (אחרי השינויים, לפי מעבד)

מכונה ב-DigitalOcean	ליבות / זיכרון	לחודש	תהליכי שרת	ילדים בשיעור בו-זמנית
(היום) s-2vcpu-2gb	2 / 2GB	\$18	1	כ-176
s-2vcpu-4gb	2 / 4GB	\$24	1-2	כ-176-350
s-4vcpu-8gb	4 / 8GB	\$48	3	כ-530
(ליבות ייעודיות) c-4	4 / 8GB	\$84	3	כ-530, יציב יותר
s-8vcpu-16gb	8 / 16GB	\$96	7	כ-1,240
(ליבות ייעודיות) c-8	8 / 16GB	\$168	7	כ-1,240, יציב
(ליבות ייעודיות) c-16	16 / 32GB	\$336	15	כ-2,650

תהליך אחד לכל ליבה, ליבה אחת נשמרת ל-worker של המדיה, nginx ומערכת ההפעלה. מחירים מדף המחירים של DigitalOcean, 25.09.2026 (חיוב לפי שנייה). ליבות Basic זולות יותר אבל משותפות עם שכנים; CPU-Optimized ייעודיות — לשירות שהמגבלה שלו מעבד, זה זמן תגובה צפוי. השרת של היום מריץ גם Cursor, MongoDB, Docker, Claude-ו (כ-580MB), עם כ-780MB פנויים ו-1GB כבר ב-swap. פרודקשן לא צריך לחלוק מכונה עם כלי פיתוח.

100,000 ילדים בו-זמנית — מה זה אומר

- כ-566 ליבות עסוקות של שרת (כ-80 מכונות c-8, כ-\$13,000 לחודש) — או פי 3-5 פחות אחרי המשימות ברשימה.
- כ-21,000 קריאות לבסיס הנתונים בשנייה. Supabase נתן כ-600-1,000 בשנייה במדידה של 10.09 — צריך מחשוב גדול בהרבה, העתקי קריאה, והרבה פחות קריאות לילד.
- כ-2,500-5,000 קריאות למודלים בשנייה — מעבר למגבלות ברירת המחדל של הספקים. הערכה גסה (1,500 טוקנים פנימה / 300 החוצה): כ-\$0.20 לילד לשעה, כלומר כ-\$20,000 לשעה ב-100k. צריך למדוד מ-ai_calls לפני שמתכננים.

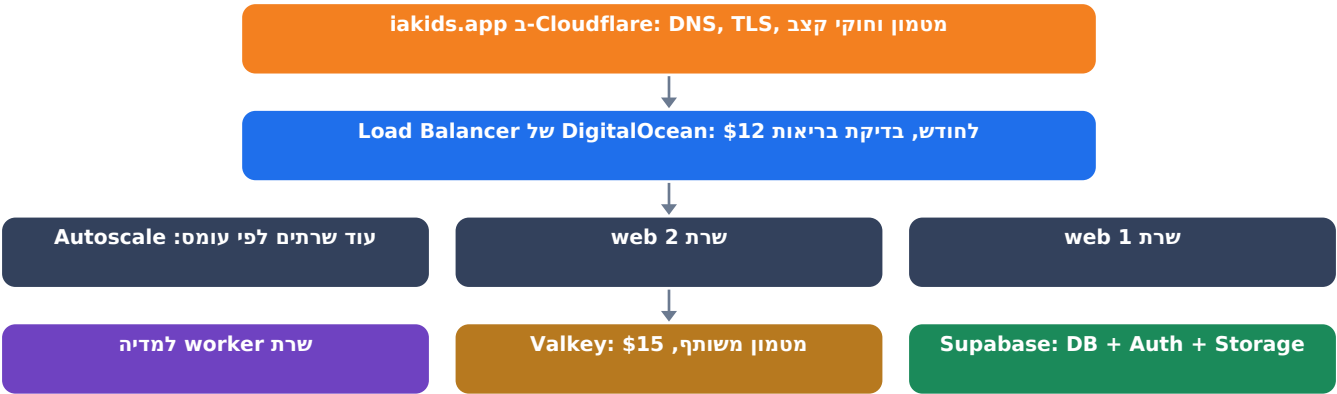
ארכיטקטורה: להעביר את ה-API הכבד לשרת אחר?

יש בשרת שלושה סוגי עבודה שונים מאוד, והם מפריעים אחד לשני כשהם על אותה מכונה.

סוג	דוגמאות	מעבד לבקשה	מחכה ל-	כמה לילד
בדיקות סטטוס וקריאות	visuals, audio, active-lesson-state, דפי ילד	20-5 מ"ש	בסיס הנתונים	גבוה (כ-4 בדקה)
נתיבי AI	תשובות בשיעור, צ'אט, שיעורי בית, הקראה	126-80 מ"ש	המודל, 20-2 שני	בינוני (כ-3 בדקה)
יצירת מדיה	תמונות, סרטוני פתיחה, קול השיעור	פרצים כבדים, כ-200MB	מודלי תמונה/קול, 5-1 דק'	לשיעור, לא לילד

היום זה נראה: עומס הבדיקה על אותה מכונה העלה את ה-p95 של השירות החי מ-12 ל-660 מ"ש.

איך מחברים הרבה שרתים לאותו שם (DNS)



רשומת DNS אחת מצביעה על ה-Load Balancer (או Cloudflare מעביר אליו). השרתים מאחוריו נמצאים לפי תגית, כך ש-Autoscale Pool יכול להוסיף ולהוריד שרתים לפי מעבד (למשל יעד 60%). דרוש: snapshot עם שירותי systemd, מפתחות SSH, ושרת בלי מצב פנימי. שים לב: חשבון DigitalOcean חדש מוגבל ל-3 מכונות — צריך לבקש הגדלה מראש.

ההמלצה, לפי הסדר — והאם זה טוב

מה	האם כדאי	איך
א המדיה לשרת משלו worker	כן, עכשיו. סיכון נמוך	אותו קוד, systemd על מכונה שנייה (\$24). בלי שינוי קוד
ב שרת בלי מצב פנימי	חובה לפני תהליך/שרת שני	או לטבלה Valkey-ומגביל הקצב ל EXAM_SETS
ג כמה שרתי web מאחורי שם אחד	כן, אחרי ב	Load Balancer + Autoscale Pool
ד נתיבי AI לקבוצת שרתים משלהם	כן, כששרת אחד לא מספיק	חוק ניתוב לפי נתיב, בלי שינוי קוד
ה יצירת שיעור חדש לתור	כן	אף בקשה לא מחזיקה 90 שני של מודל
ו תוכן השיעור ב-Cloudflare	כן, פי כמה	התוכן זהה לכל הילדים: JSON עם גרסה
ז דחיפה במקום בדיקות חוזרות	כן	או המתנה ארוכה יותר: כ-שליש מהבקשות SSE
ח שרת אחד ענק	לא	תקרה כ-2,650 ילדים ונקודת כשל יחידה

רשימת משימות — מדורגת לפי מה שהמידדות אומרות

כל שורה עם הסיבה הנמדדת שלה.

#	משימה	למה (נמדד)	מאמץ
1	לבדוק ולהעלות את השינויים של היום	פי 2.5-6 בקריאות, -54% קריאות DB, הצ'אט לא מקפיא	מוכן, מחכה לאישור
2	(המדיה לשרת משלו \$24 worker	עומס על המכונה המשותפת: 12 ← 660 מ"ש לשירות החי	שעה, בלי קוד
3	כלי פיתוח (Cursor, Claude, Mongo, Docker) מחוץ לשרת הפרודקשן	כ-580MB מתוך 1GB, 2GB ב-swap	תפעול
4	EXAM_SETS ל-Valkey/DB ומגביל הקצב	חוסם תהליך או שרת שני	חצי יום
5	פחות קריאות DB לתשובה בשיעור (10 ← 3-2)	נתיבי AI הם רוב המעבד לילד (126 מ"ש)	יום
6	לקוח DB אסינכרוני בנתיבים החמים	כ-3-4 מ"ש מעבד לכל קריאת רשת	2-3 ימים
7	שדרוג ל-\$48 (s-4vcpu-8gb) עם 3 תהליכים (אחרי 4)	כ-176 ← כ-530 ילדים	שעה + הפעלה מחדש
8	הגדלת; Autoscale + שרתים 2 + Load Balancer מגבלת החשבון	בלי נקודת כשל יחידה	יום
9	יצירת שיעור לתור	אין בקשה של 70-90 שני	1-2 ימים
10	תוכן שיעור כ-JSON בקצה של Cloudflare	מוריד את רוב הקריאות של מסך השיעור	2-3 ימים
11	המתנה ארוכה במקום בדיקה כל 1.5-1 שני / SSE	כ-שליש מהבקשות של ילד	1-2 ימים
12	נתיבי AI לקבוצת שרתים משלהם	צ'אט עמוס לא מאט את הבדיקות	חוק ניתוב, אחרי 8
13	מחשוב גדול יותר + העתקי קריאה; מדידה: Supabase מכמה מכונות	21,000 קריאות/שני ב-100k	תכנון + תקציב
14	תוכנית עלות וספקי AI (מטמון, מכסות, הגדלת מגבלות)	כ-\$0.20 לילד לשעה בהערכה	עסקי
15	ניסיון חוזר אחד על חיבור שנפל ל-Supabase	נראה ב-14.09 ובבדיקה "Server disconnected" שעה היום	
16	פחות לוגים לכל בקשה בנתיבים החמים	כ-3% מהמעבד	שעה

DigitalOcean: API, MCP, Autoscale

- התראות, יתרה, Load Balancers, רשימת מכונות, שינוי גודל (OAuth): התחברות, droplets.mcp.digitalocean.com, רשמי קיים MCP וחשבוניות. אין בו גרף מעבד/זיכרון ואין Autoscale Pools — לאלה ה-API.
- שינוי גודל. do-agent קורא מדדי מעבד וזיכרון (דורש droplet:read, monitoring:read, billing:read) לקריאה בלבד API: token מכבה את המכונה — השבתה קצרה.
- הוא הדרך App Platform. Load Balancer יעד מעבד/זיכרון, מינימום/מקסימום (מאחורי) Droplet Autoscale Pools — כן: Autoscale עם הכי פחות תפעול.

מה יקרה בהעלאה, ומה נבדק

- קבצים: request_cache.py (+70/-16), backend-ai-tutor-he/main.py, tools/prompt_gate.py, תיקיית /performance חדשה, שורה ב-CLAUDE.md. בלי מיגרציה, בלי תלות חדשה, בלי שינוי ב-frontend.
- אחרי deploy: כניסה נבדקת מקומית; שורות ילד/מנוי/תוכנית עד דקה (או 5 דק') ישנות; שלושת הנתיבים מפסיקים להקפיא את השרת.
- ה-gate עובר, וכל חוק חדש נבדק שהוא נכשל כשמקלקלים: נתיב בלי בדיקת ביצועים, המתנה לרשת בתוך לולאת השרת, חריגה מתקציב קריאות DB, מטמון שהוסר, token פג / מזויף / לקהל אחר / מהסוג הישן, שגיאת תחביר ב-main.py.
- לא הועלה כלום: אין commit, push או deploy עד שתאשר.